

Drug Tracking System

Data Engineering Pipeline

Assumptions, Responsibilities & Integration Guide

DEPI Graduation Project — Team: TheSilence_DEPI

April 2026

1. Document Purpose

This document captures all assumptions made during the design and implementation of the Drug Tracking System data pipeline. It also defines the responsibilities of each team and outlines what is required from each team for the pipeline to function correctly.

This document is intended for:

- The Data Engineering team (pipeline owner)
- The Backend/API team
- The Frontend team
- The Database/DevOps team
- Graduation project discussion and review

2. System Overview

The Drug Tracking System is a web-based platform that allows patients to search for available drugs and find the nearest pharmacies that have them in stock. The data engineering pipeline is responsible for ingesting, transforming, and delivering clean data to the warehouse.

2.1 Pipeline Architecture

The full pipeline flow is:

Stage	Tool	Location	Description
Data Lake	File System	Azure VM	Raw CSV files from pharmacies land here
Normalization	Python (column_normalizer.py)	Azure VM	Standardizes column names using fuzzy matching
Transformation	DBT + DuckDB	Azure VM	Cleans, validates, and builds all models
Warehouse	Azure PostgreSQL	Azure Cloud	Final destination for all clean tables
Orchestration	Apache Airflow	Azure VM	Automates the full pipeline every 12 hours

3. All Assumptions Made

The following assumptions were made during the design and implementation of the pipeline. Each team must be aware of these and confirm or challenge them before production deployment.

3.1 Database Assumptions

- Azure PostgreSQL is used as the data warehouse (destination only — not for transformation)
- A database named 'medical_system' already exists in the Azure PostgreSQL server
- All 7 pipeline tables are created in the 'public' schema of medical_system
- The pharmacies table and patients table already exist in PostgreSQL and are managed by the backend team — the pipeline assumes they are available
- The pharmacy_id values used in drug_prices CSV files already exist in the pharmacies table (managed by backend)

3.2 Data Model Assumptions

- drug_prices table serves as the pharmacy inventory — it was renamed from pharmacy_inventory to avoid confusion with the original schema
- The quantity column was intentionally removed from the inventory — the system only tracks availability (true/false) and price, not exact stock counts
- drug_id values in drugs.csv are trusted and pre-assigned — the pipeline does NOT auto-generate drug IDs for existing drugs
- category_id values in drug_categories.csv are trusted and pre-assigned — the pipeline does NOT auto-generate category IDs
- If a pharmacy uploads a drug name that does not exist in the main drugs table, it is automatically added with minimal information and a new non-conflicting drug_id
- drug_alternatives relationships are provided by the data team — not extracted from pharmacy uploads

3.3 CSV File Assumptions

- Structured CSVs (drugs.csv, drug_categories.csv, drug_alternatives.csv) are maintained by the data team with correct column names and IDs — no normalization needed
- Pharmacy inventory CSVs (drug_prices) come from pharmacies with unknown column names — full normalization via column_normalizer.py is applied
- Pharmacy inventory CSVs contain drug NAMES (not drug IDs) — the pipeline resolves drug_id internally by joining on name
- The naming convention for pharmacy uploads is:
drug_prices_pharmacy_<id>_<timestamp>.csv (e.g.,
drug_prices_pharmacy_5_20260425_103000.csv)
- Multiple pharmacy uploads can exist simultaneously in the datalake — they are all merged into one seeds/drug_prices.csv before DBT runs
- If two records for the same pharmacy/drug pair exist across multiple uploads, the most recent one is kept

3.4 Pipeline Execution Assumptions

- The pipeline runs every 12 hours automatically via Airflow (at midnight and noon)

- Manual triggers are safe — running the pipeline multiple times does not create duplicate data
- Core tables (drugs, drug_categories, drug_alternatives, drug_prices, drug_analytics) are truncated and rebuilt on every run
- History tables (price_history, availability_history) are append-only — only new records are inserted, existing history is never deleted
- DuckDB 1.2.1 is used in both dbt_env and airflow_env to ensure version compatibility
- Airflow 3.x is used — deprecated import paths for BashOperator and PythonOperator have been updated to airflow.providers.standard

3.5 File Transfer Assumptions

- The web application and the pipeline VM are on separate servers
- Azure Blob Storage is the agreed intermediary for file transfer between the backend server and the pipeline VM
- The backend API uploads pharmacy CSVs to Azure Blob Storage after receiving them from the pharmacy
- The pipeline VM downloads new CSVs from Azure Blob Storage to the datalake folder at the start of each pipeline run
- This Blob Storage download step needs to be added as Step 0 to the Airflow DAG before going to production

4. Team Responsibilities

4.1 Data Engineering Team (Pipeline Owner)

Responsibility	Status
Column normalization for pharmacy CSV uploads	✓ Done
DBT staging models (stg_drugs, stg_drug_categories, stg_drug_alternatives, stg_drug_prices)	✓ Done
DBT core models (core_drugs, core_drug_categories, core_drug_alternatives, core_drug_prices)	✓ Done
DBT analytics model (drug_analytics)	✓ Done
DBT snapshot models (price_history, availability_history)	✓ Done
Push to PostgreSQL script	✓ Done
Airflow DAG orchestration	✓ Done
Systemd auto-start on boot	✓ Done
Azure Blob Storage download step (Step 0 in DAG)	⌚ Pending

Responsibility	Status
Deploy pipeline to Azure VM	 Pending

4.2 Backend Team Requirements

The following is required from the backend team for the pipeline to work correctly:

Requirement	Details
CSV naming convention	Pharmacy inventory uploads must follow: <code>drug_prices_pharmacy_<id>_<timestamp>.csv</code>
Azure Blob Storage integration	Backend API must upload pharmacy CSVs to the agreed Azure Blob Storage container after receiving them
pharmacy_id consistency	The pharmacy_id in uploaded CSVs must match the pharmacy_id in the pharmacies table in PostgreSQL
Registration & login	Handled entirely by backend — direct write to PostgreSQL, no pipeline involvement
Manual inventory updates	Single row insert/update/delete goes directly to drug_prices table via API — no pipeline needed
Pharmacies table	Must exist in PostgreSQL medical_system database before pipeline runs — pipeline assumes it is available
Users table	Must exist in PostgreSQL — managed entirely by backend

4.3 Frontend Team Requirements

Requirement	Details
CSV upload UI	Must allow pharmacy to upload a CSV file and send it to the backend API endpoint
Drug search	Must query drug_prices and drugs tables for availability and pricing
Analytics display	Must read from drug_analytics table for avg/min/max price and availability percentage
Pharmacy profile	Must read from pharmacies table (backend-managed)
Favorites & notifications	Backend-managed — no pipeline involvement

4.4 Database/DevOps Team Requirements

Requirement	Details
Azure PostgreSQL server	Must be accessible from both backend server and pipeline Azure VM

Requirement	Details
medical_system database	Must exist — already created
Firewall rules	Pipeline VM IP must be whitelisted in Azure PostgreSQL firewall
Azure Blob Storage container	Must be created and shared access configured for both backend and pipeline VM
Azure VM	Must be Linux-based with Python 3.10+ available

5. Pipeline Tables Reference

The following 7 tables are owned and managed by the data pipeline:

Table	Source	Strategy	Purpose
drug_categories	drug_categories.csv (data team)	TRUNCATE → INSERT	Master category reference
drugs	drugs.csv (data team)	TRUNCATE → INSERT	Master drug reference — auto-adds new drugs from pharmacy uploads
drug_alternatives	drug_alternatives.csv (data team)	TRUNCATE → INSERT	Maps each drug to its alternatives
drug_prices	Pharmacy CSV uploads	TRUNCATE → INSERT	Pharmacy inventory — available drugs and prices per pharmacy
drug_analytics	Computed by DBT	TRUNCATE → INSERT	Avg/min/max price and availability % per drug
price_history	DBT snapshot of drug_prices	Append only	Full history of price changes over time
availability_history	DBT snapshot of drug_prices	Append only	Full history of availability changes over time

6. What Is NOT the Pipeline's Concern

The following operations are handled by the backend team directly and do not involve the pipeline:

- User registration — backend writes directly to PostgreSQL users table
- User login and session management — backend handles authentication
- Role-based access control — backend enforces permissions

- Single-row manual inventory updates by pharmacies — backend API writes directly to drug_prices table
- Patient favorites — backend manages favorites table
- Notifications — backend manages notifications table
- Ratings — backend manages ratings table
- Search logs — backend manages search_logs table
- Password encryption — backend responsibility

7. Pending Items Before Production Deployment

Item	Owner	Priority
Add Azure Blob Storage download step to Airflow DAG	Data Engineering	● High
Deploy pipeline to Azure VM	Data Engineering	● High
Backend API integration with Azure Blob Storage	Backend Team	● High
Confirm pharmacy_id consistency between CSV uploads and pharmacies table	Backend + Data Engineering	● High
Create Azure Blob Storage container and configure access	DevOps	● High
Whitelist Azure VM IP in PostgreSQL firewall rules	DevOps	● Medium
Agree on final CSV naming convention with backend team	Backend + Data Engineering	● Medium
Test pipeline end-to-end on Azure VM with real data	Data Engineering	● Medium

8. Environment Setup Summary

8.1 Virtual Environments

Environment	Tool	Key Packages
dbt_env	DBT + DuckDB	dbt-duckdb 1.9.2, duckdb 1.2.1
airflow_env	Airflow + Pipeline scripts	airflow 3.1.1, psycopg2-binary, pandas, rapidfuzz, duckdb 1.2.1

8.2 Key Configuration Files

File	Location	Purpose
profiles.yml	~/.dbt/profiles.yml	DBT connection to DuckDB
dbt_project.yml	dbt_architecture/drug_tracking/	DBT project configuration
airflow.cfg	airflow_env/	Airflow configuration (AIRFLOW_HOME)
airflow.service	/etc/systemd/system/	Auto-start Airflow on system boot
column_normalizer.py	Medical_System_Pipeline/	CSV column normalization
push_to_postgres.py	Medical_System_Pipeline/	Push DuckDB tables to PostgreSQL
drug_tracking_pipeline.py	Medical_System_Pipeline/dags/	Main Airflow DAG

9. Notes for Graduation Discussion

Key points to highlight during the graduation project discussion:

Why DuckDB instead of PostgreSQL for transformation?

- DuckDB reads CSV files natively without a pre-loading step
- Transformations happen locally on the VM — very fast
- PostgreSQL remains clean — only receives final structured tables
- Clear separation: VM = transformation layer, PostgreSQL = serving layer

Why two virtual environments?

- Prevents package version conflicts between DBT and Airflow
- DBT commands run in dbt_env via BashOperator
- Python scripts (normalizer, push to postgres) run in airflow_env via PythonOperator

Why local VM first before Azure VM?

- Protects Azure credits — all debugging done locally for free
- Local Linux VM mirrors Azure Linux VM — same OS, same commands
- Only connection strings and paths change when deploying to Azure

Why snapshots for price_history and availability_history?

- Snapshots are append-only — history is never deleted
- Every price or availability change is recorded with a valid_from/valid_to timestamp
- Supports FR-12: notify patients of price changes

- Supports FR-9: suggest alternatives when drug available in fewer than 3 pharmacies
- Supports FR-19: notify patient when previously unavailable drug is back

Why fuzzy matching in `column_normalizer`?

- Pharmacies upload CSVs with unknown column names
- Exact matching handles known variations
- Fuzzy matching (rapidfuzz) handles typos and close variations
- Blacklist prevents generic columns like 'id' from being mismatched